

Introduction à Linux embarqué

Mylène Josserand

(ex) Développeuse et formatrice Linux embarqué

Cours IUT, Mars 2026





- ▶ Master pro génie physiologique et informatique, option Imagerie Médicale
- ▶ Ancienne Ingénieure Noyau Linux et formatrice (10 ans)
- ▶ Différentes entreprises : Navocap, Smile, Bootlin, Collabora
- ▶ Développement de drivers Linux, intégration système sous Yocto Project et Buildroot
- ▶ Maintenant, éducatrice comportementaliste canin et félin (Rivière Positive)
- ▶ `josserand.mylene@gmail.com`



Dessin issu d'une photographie de Samuel Blanc

DISCLAIMERS

Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



S'il vous plait

- ▶ Tutoyez moi !
- ▶ Mais surtout, ne m'appellez pas Madame mais Mylène xD



Dessin issu d'une photographie de Samuel Blanc

Généralités

Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



Naissance de logiciel libre

- ▶ 1983: Richard Stallman - Projet **GNU** et concept de logiciel libre. Développement de gcc, gdb, glibc, etc
- ▶ **1991**: Linus Torvalds - Projet noyau (**kernel**) **Linux**. De type Unix
- ▶ 1995: Linux est de plus en plus populaire sur les serveurs
- ▶ 2000: Linux est de plus en plus populaire sur les **systèmes embarqués**
- ▶ 2008: Linux est de plus en plus populaire sur les appareils mobiles
- ▶ 2010: Linux est de plus en plus populaire sur les téléphones



Logiciel libre ?

- ▶ Un logiciel est considéré libre quand sa license respecte ces **4 libertés**:
 - ▶ Liberté d'*exécuter* le logiciel pour n'importe quel but
 - ▶ Liberté d'*étudier* le logiciel et de le *modifier*
 - ▶ Liberté de *redistribuer* des copies
 - ▶ Liberté de *distribuer* des copies modifiées
- ▶ Le code doit être disponible, le logiciel peut être modifié et distribué aux utilisateurs

Parfait pour les systèmes embarqués !



Historique de Linux

- ▶ Créé par Linus Torvalds en 1991
- ▶ Est un noyau : logiciel gérant les ressources d'une machine
- ▶ N'est pas un système d'exploitation : est lié au projet GNU
→ distribution GNU/Linux





L'intérêt pour les entreprises est:

- ▶ de maîtriser les sources de leur OS
- ▶ d'économiser le prix des licences
- ▶ de bénéficier du support d'une communauté importante de développeur
- ▶ d'utiliser des composants déjà existants et testés par d'autres personnes

Seule la partie Desktop a du mal face à ses concurrents



Pourquoi Linux ?

- ▶ Gratuit, librement modifiable et distribuable
- ▶ Enorme support matériel
- ▶ Très flexible : Convient aux systèmes très contraints comme très puissants
- ▶ Très bon écosystème, beaucoup de manières d'avoir un accompagnement professionnel



Dessin issu d'une photographie de Samuel Blanc

Architecture GNU/Linux

Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



Architecture





Organisation d'un rootfs

- ▶ Organisation commune à 90% entre les UNIX
- ▶ Quelques spécificités GNU/Linux et distribution
- ▶ Tout commence depuis la racine /
- ▶ Puis, organisation en dossier/sous-dossier:
 - ▶ /bin,/sbin,/usr/bin,/usr/sbin: binaires communs et systèmes
 - ▶ /lib,/usr/lib: bibliothèques et modules noyau
 - ▶ /etc: fichiers de configuration
 - ▶ /dev: nœuds d'accès aux périphériques
 - ▶ /var: fichiers variables comme log, mail, ...
 - ▶ /opt: pour les programmes externes (ex: LibreOffice)
 - ▶ /home: accueille les répertoires des utilisateurs



Organisation d'un rootfs

- ▶ Quelques répertoires spéciaux:
 - ▶ `/lib/modules`: contient les modules du noyau
 - ▶ `/root`: home-directory de l'utilisateur root
 - ▶ `/media`: point de montage des volumes amovibles
 - ▶ `/proc`: système de fichier virtuel (état du système)
 - ▶ `/sys`: idem pour les périphériques connectés
 - ▶ `/boot`: noyau statique (vmlinuz, ulmage, ...)



Démo avec /sys

- ▶ Tout est fichier et /sys permet de faire des modifications avec le matériel :
 - ▶ Se mettre en root

```
sudo su
```

- ▶ Allumer la led du CAPS LOCK

```
echo 1 > /sys/class/leds/input0\:\:capslock/brightness
```

- ▶ Éteindre la led du CAPS LOCK

```
echo 0 > /sys/class/leds/input0\:\:capslock/brightness
```

- ▶ Ou on peut aussi faire que la led s'allume à chaque paquet Ethernet reçu !

```
echo phy0tx > /sys/class/leds/input0\:\:capslock/trigger
```

- ▶ Et pour revenir comme avant (sinon, c'est difficile d'écrire vu que vous aurez des majuscules n'importe quand !) :

```
echo none > /sys/class/leds/input0\:\:capslock/trigger
```



Dessin issu d'une photographie de Samuel Blanc

Execution d'un système GNU/Linux

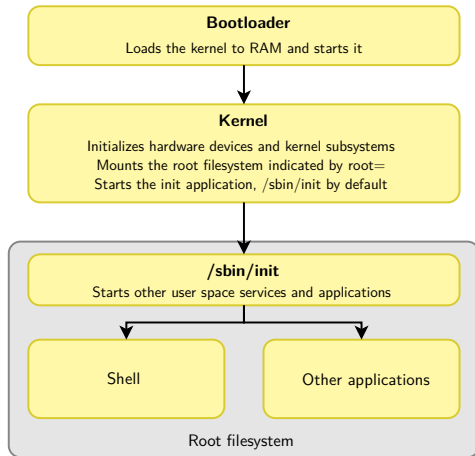
Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



Démarrage

1. Le matériel lance le bootloader
2. Le bootloader configure la RAM et le support contenant le noyau
3. Le bootloader copie le noyau en RAM et lui donne la main
4. Le noyau s'auto extrait en RAM
5. Le noyau initialise les périphériques et les sub-systèmes (ordonnanceur, pile réseaux, système de fichier,...)
6. Le noyau récupère dans le rootfs le binaire d'initialisation (`/sbin/init` par défaut) et crée le processus 1
7. Ce processus est chargé de démarrer tous les services en espaces utilisateurs





Dessin issu d'une photographie de Samuel Blanc

Systèmes d'exploitation embarqués

Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



Qu'est-ce que Linux embarqué?

Linux Embarqué est l'utilisation du **kernel Linux** et différents composants **open-source(GNU)** dans des **systèmes embarqués**



- ▶ Système répondant à des **besoins** et des **contraintes**
- ▶ **dédié** à une classe de cas d'utilisations
- ▶ Lorsque les besoins incluent la nécessité d'effectuer des calculs et traitements de données, inclut un **processeur**
- ▶ Pour les systèmes informatiques embarqués, on considère souvent 3 aspects :
 - ▶ Mécaniques et Environnementaux : Interaction avec l'environnement
 - ▶ Électriques et Électroniques : Alimentation, Gestion des Entrées/Sorties
 - ▶ Logiciels : Support du matériel, Logique métier



Exemples





Linux dans l'embarqué

- ▶ Linux est un **noyau**, il n'est qu'une pièce de la **pile logicielle**
- ▶ Linux peut être utilisé sur des processeurs très peu puissants, comme des super-calculateurs
- ▶ Il faut tout de même quelques Méga-octets de **Mémoire Vive** et de **Mémoire Morte**
- ▶ Linux peut tourner sur certains micro-contrôleurs puissants
- ▶ Mais on le retrouve surtout sur les micro-processeurs et SoCs
- ▶ De nombreuses architectures CPU sont supportées : x86, ARM, RISC-V, MIPS, Power, ia64, SuperH...
- ▶ Beaucoup de fonctionnalités avancées sont présentes : Réseau, Sécurité, Virtualisation, Filesystems, etc.

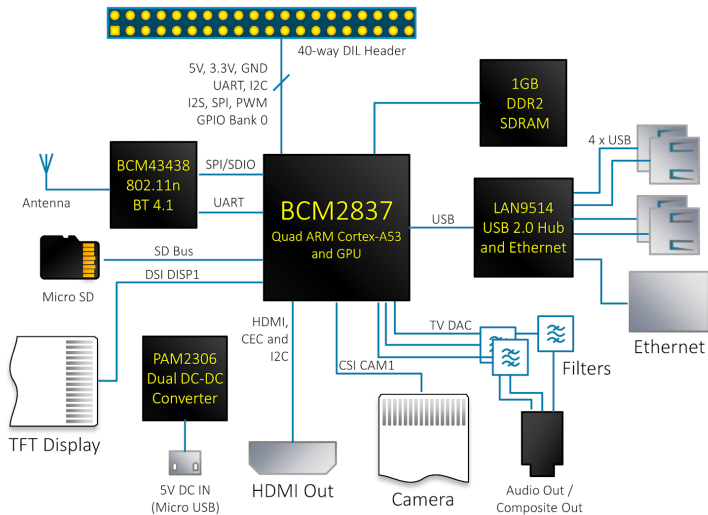


Architecture typique d'un système informatique

- ▶ Un ou plusieurs **CPU** : *Central Processing Unit*
- ▶ De la mémoire vive (**RAM** : *Random Access Memory*)
- ▶ De l'électronique de gestion : Alimentation, horloges, etc.
- ▶ Des périphériques :
 - ▶ Mémoire persistante : Disque dur, eMMC, Flash, etc.
 - ▶ Entrées/sorties : Clavier, **GPIO**, écran, caméra, etc.
 - ▶ Réseau : Ethernet, Wifi, Bluetooth, etc.
 - ▶ Internes : Alimentation, gestion de la température, etc.
 - ▶ **Bus** génériques : USB, PCI, etc.



Exemple : Raspberry pi 3





Dessin issu d'une photographie de Samuel Blanc

Construire son système embarqué

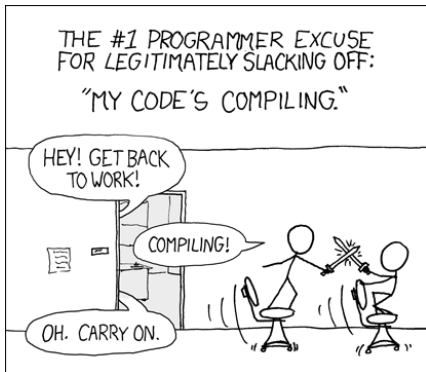
Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



Avant de nous lancer encore sur de la théorie...

- ▶ On va faire un peu de pratique !
- ▶ Je vous propose de passer sur le lab 1 et on continuera de papoter pendant que ça compile





Éléments nécessaires

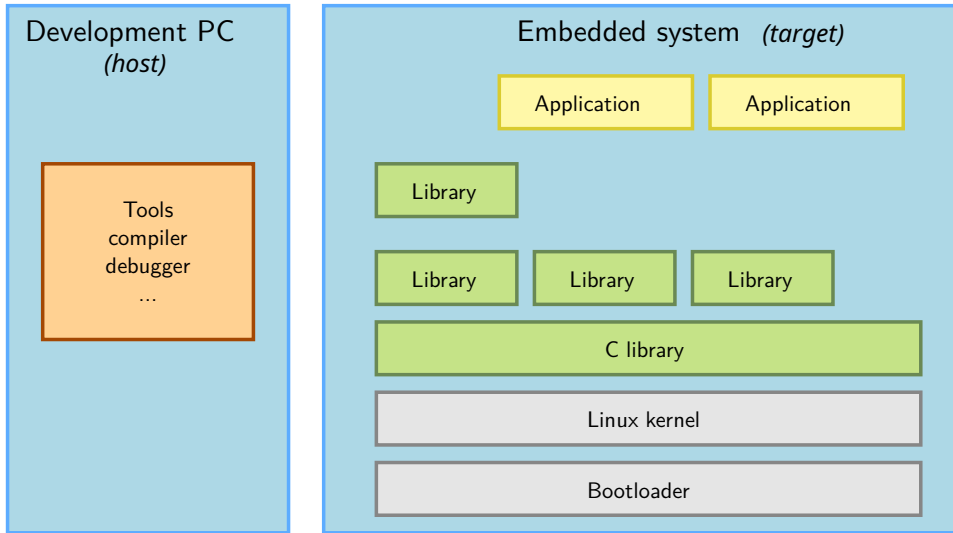


Composants software

- ▶ Toolchain de cross-compilation
 - ▶ Compilateur qui est exécuté sur la machine de développement mais qui génère du code pour la cible
- ▶ Bootloader
 - ▶ Démarre le hardware, responsable des initialisations basiques, charge et exécute le kernel
- ▶ Kernel Linux
 - ▶ Contient la gestion des processus et de la mémoire, le réseau, les pilotes des périphériques et fournit des services aux applications user space
- ▶ Librairie C
 - ▶ Interface entre le kernel et les applications user space
- ▶ Bibliothèques et applications
 - ▶ De tiers ou faites maisons



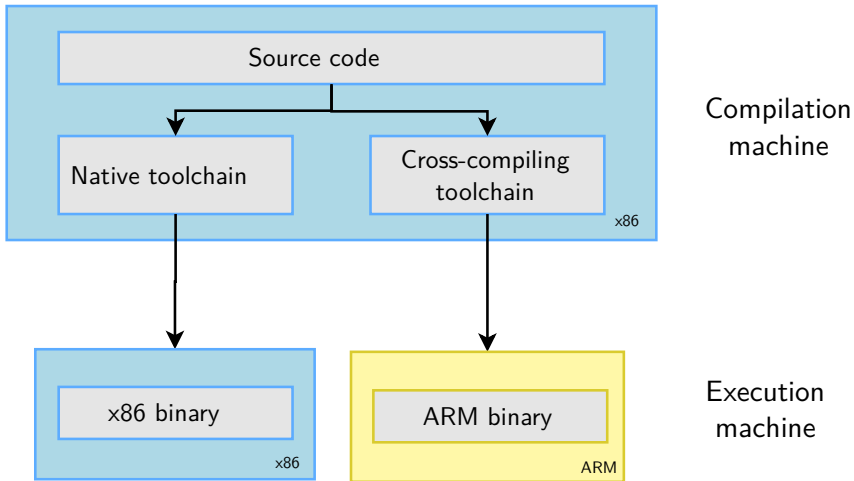
Architecture globale





Toolchain

- Un point très complexe !
- A besoin du compilateur pour la cible et pas sur le pc de développement





Toolchain - pour simplifier

- ▶ Votre PC de développement (x86)
- ▶ Une Playstation 5 donc vous avez besoin de CD de playstation pour jouer
- ▶ Votre Raspberry Pi (ARM)
- ▶ Une Game boy donc vous avez besoin de cartouches de Game Boy pour jouer





Bootloader



Bootloader

- ▶ Premier logiciel lancé au démarrage de la machine
- ▶ Le Noyau Linux étant relativement gros, il ne rentre souvent pas dans la **RAM**
- ▶ **Initialise le matériel** (RAM, stockage, ...) nécessaire au boot
- ▶ **Charge le noyau** en RAM et lui donne la main
- ▶ Permet de donner des arguments au noyau
- ▶ Exemples:
 - ▶ **U-boot** (supporte de nombreuses architectures, la référence)
 - ▶ **Barebox** (u-boot v2, meilleure archi, moins de fonctionnalités pour l'instant)
 - ▶ grub (x86, serveur et desktop principalement)
 - ▶ isolinux (alternative x86 à grub)
- ▶ Un système embarqué nécessitera une configuration et un support matériel dans le bootloader.

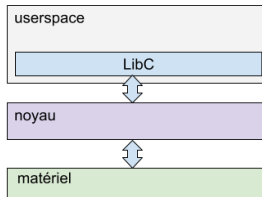


Noyau alias kernel



Rôle du Noyau

- ▶ Noyau monolithique (en un seul fichier)
- ▶ Initialise le matériel au démarrage
- ▶ Gère les périphériques via des **pilotes (drivers)**
- ▶ Gère la mémoire en attribuant des **Plages d'adresse virtuelles** aux processus
- ▶ Gère l'utilisation du CPU par les processus via son **ordonnanceur**
- ▶ Création des processus via `fork()` et `exec()` : multi-threading
- ▶ Nombreuses piles réseau (IPv4, IPv6, Ethernet, etc.)
- ▶ Fournit des **couches d'abstraction** à l'espace utilisateur
- ▶ Il existe de nombreux noyaux utilisables dans l'embarqué. La Playstation 4 utilise une variante de FreeBSD.





- ▶ Le noyau Linux est développé très largement en C (31 000 fichiers .c)
- ▶ On y trouve aussi un peu d'Assembleur pour des parties très bas-niveau
- ▶ Certains projets récents permettent de développer des parties en Rust
- ▶ Il s'agit d'un très gros projets, de plusieurs millions de lignes de code
- ▶ Plusieurs milliers de contributeurs ajoutent et corrigent des fonctionnalités chaque mois
- ▶ Il se compile via un **makefile** et se configure via **Kconfig**

Mes contributions



Device-tree

- ▶ Linux doit savoir quels périphériques sont présents sur le matériel
- ▶ Il doit aussi savoir leur configuration matérielle : Fréquence, bus, adresse, etc.
- ▶ Il doit enfin pouvoir déterminer quel pilote utiliser pour chacun des périphériques
- ▶ Dans l'embarqué, on utilise le **device-tree**. Il s'agit d'un fichier décrivant le matériel



Device-tree - 2

- ▶ Le Device-Tree est un fichier texte `.dts` contenant une description du matériel
- ▶ Il est compilé en `.dtb`, format reconnu par le noyau
- ▶ Le Bootloader charge le **dtb** en RAM avec le noyau au démarrage
- ▶ Le noyau lit le dtb, et charge les pilotes en conséquence
- ▶ Chaque modèle de carte électronique a son propre dts
- ▶ De nombreux dts pré-écrits peuvent être trouvés dans les sources du noyau :
`bcm2710-rpi-3-b.dts` pour RPi3, `bcm2711-rpi-4-b.dts` pour RPi4



Rootfs



Binaires de base

- ▶ Binaires de base indispensables au système (ls, bash, dd, top, cp, mv, init,...)
- ▶ Trois possibilités:
 - ▶ Récupérer les sources et les compiler un par un →laborieux
 - ▶ GNU/Linux coreutils →trop lourd pour l'embarqué
 - ▶ **Busybox** : regroupe la majorité des commandes Linux en un seul executable
 - ▶ 95% des distributions Linux embarqués l'utilise
 - ▶ Simple, léger, portable
 - ▶ Diffusé sous licence GPLv2



Rootfs - Peuplement

► Peuplement d'un rootfs:

1. Installation de la libc et des autres librairies fournies par la toolchain
2. Installation des binaires de busybox
3. Création de la configuration de démarrage (/etc/init.d)
4. Ajout des nodes dans /dev (si pas de mécanisme automatique)
5. Ajout des librairies et applications du projet.

► Deux méthodes:

- A la main, copier ou installer chaque fichier/logiciel manuellement → Inenvisageable à moyen et long terme
- Automatiser soit en scriptant soit en utilisant un système existant (Yocto, buildroot, ...) → Plus rapide et plus sûr. Indispensable en production pour maîtriser son livrable



Dessin issu d'une photographie de Samuel Blanc

Résumé

Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



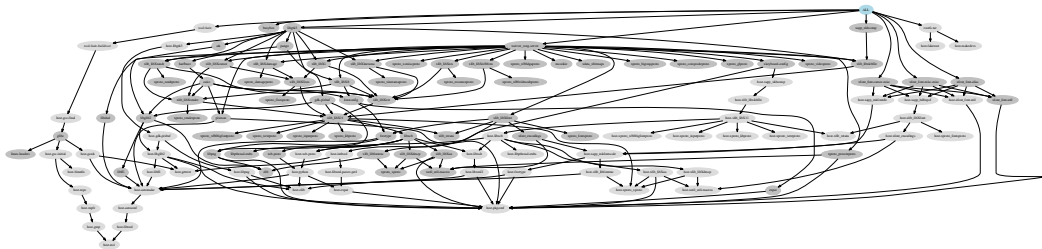
Résumé

- ▶ Les systèmes embarqués basés sur l'écosystème Linux nécessitent de nombreux composants
- ▶ Un Bootloader, potentiellement composé de plusieurs programmes : Pour le démarrage
- ▶ Le Noyau Linux : Pour la gestion du matériel et des ressources
- ▶ Une LibC : Pour interagir avec le noyau
- ▶ Un init-system : Pour démarrer et gérer les programmes en userspace
- ▶ Des outils courants : Pour utiliser le système et développer des programmes complexes
- ▶ Les application métier !
- ▶ Une chaîne de compilation : Pour compiler tout ça !



Automatiser la génération d'un système

- ▶ Beaucoup des composants nécessaires sont open-source, on peut facilement les compiler
- ▶ On peut choisir d'utiliser une distribution toute prête pour gagner du temps !
- ▶ Pour les systèmes embarqués, on choisit souvent de tout recompiler
- ▶ La configuration et compilation de tous les composants est automatisée par un **build-system**
- ▶ **Buildroot** et **Yocto/Openembedded** sont les deux principaux





Dessin issu d'une photographie de Samuel Blanc

Introduction à Buildroot

Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



Généralités sur BR



- ▶ Peut compiler une toolchain, un rootfs, un kernel, un bootloader
- ▶ **Facile à configurer**: menuconfig, xconfig, etc.
- ▶ **Rapide**: compile un rootfs en quelques minutes
- ▶ Facile à comprendre: écrit en make, bonne documentation
- ▶ **Petit**: root filesystem, débute en 2 MB
- ▶ **2200+ paquets** pour userspace
- ▶ **Pleins d'architectures** supportées
- ▶ **Technologies bien connues**: *make* et *kconfig*
- ▶ Neutre
- ▶ Communauté active, releases régulières
- ▶ <https://buildroot.org>



Buts dans le design

- ▶ Buildroot est designé avec des buts précis:
 - ▶ Simple à utiliser
 - ▶ Simple à personnaliser
 - ▶ Builds **reproductibles**
 - ▶ Petit root filesystem
 - ▶ Temps de build relativement rapide
 - ▶ Facile à comprendre
- ▶ Pas toutes les fonctionnalités possibles supportées
- ▶ Plus compliqués et avec beaucoup de fonctionnalités : Yocto Project, OpenEmbedded



Buildroot dans l'industrie?

▶ Entreprises

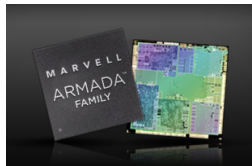
- ▶ Google
- ▶ Barco
- ▶ Rockwell Collins

▶ Vendeurs de processors

- ▶ Imagination Technologies
- ▶ Marvell
- ▶ Microchip (Atmel)
- ▶ Analog Devices

▶ Vendeur de SoM et de cartes

- ▶ Beaucoup d'entreprises lors de *R&D* sur des produits
- ▶ Beaucoup de **hobbyists** des cartes de développement : Raspberry Pi, BeagleBone Black, etc.





- ▶ Releases stables tous les 3 mois
- ▶ Archives disponibles pour chaque release stable
 - ▶ <https://buildroot.org/downloads/>
- ▶ Plus pratique via git
 - ▶ Visualisation de ses modifications
 - ▶ Simplification des contributions `git clone git://git.busybox.net/buildroot`
 - ▶ Tags git disponible pour chaque release stable
- ▶ Une **long term support (LTS)** release chaque année (maintenue 1 an)



Utilisation de Buildroot

- ▶ Implémenté via `make`
 - ▶ Avec quelques shell scripts d'aide
- ▶ Toutes les interactions via `make` dans le répertoire principal de Buildroot

```
$ cd buildroot/  
$ make help
```

- ▶ Pas besoin d'être `root`, désigné pour être exécuté avec privilèges utilisateurs
 - ▶ Exécuter en `root` est même fortement découragé!



Configuration



Configuration de Buildroot

- ▶ Comme le kernel Linux, utilise *Kconfig*
- ▶ Un choix des interfaces de configuration mais la plus simple : `make menuconfig`

```
/home/thomas/projets/buildroot/.config - Buildroot 2017.08 Configuration

Buildroot 2017.08 Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus
----). Highlighted letters are hotkeys. Pressing <Y> selects a feature, while
<N> will exclude a feature. Press <Esc><Esc> to exit, <?> for Help, </> for
Search. Legend: [*] feature is selected [ ] feature is excluded

|| Target options --->
  Build options --->
  Toolchain --->
  System configuration --->
  Kernel --->
  Target packages --->
  Filesystem images --->
  Bootloaders --->
  Host utilities --->
  Legacy config options --->

<Select>  < Exit >  < Help >  < Save >  < Load >
```



Config entière vs. *defconfig*

- ▶ `.config` = fichier de config *entier* → contient les valeurs de toutes les options
Le `.config` par défaut a +3000 lignes
→ Pas pratique et lisible
- ▶ Un `defconfig` stocke seulement les options qui n'ont pas de valeurs par défaut
→ Plus facile à lire et modifiable par des humains



Utilisation et création d'un *defconfig*

- ▶ Buildroot permet de charger un *defconfig* issu du dossier `configs/` via :
`make <foo>_defconfig`
 - ▶ Cela va surcharger le `.config`, si il y en a un
- ▶ Pour créer un *defconfig* : `make savedefconfig`



defconfigs existants

- ▶ Il existe un *defconfigs* pour différentes plateformes supportées :
 - ▶ RaspberryPi, BeagleBone Black, CubieBoard, Microchip evaluation boards, Minnowboard, quelques i.MX6 boards
- ▶ Pour les lister : `make list-defconfigs`
- ▶ La plupart des *defconfigs* sont minimaux : compile seulement une toolchain, un bootloader, un kernel et un root filesystem minimal.

```
$ make raspberrypi3_defconfig  
$ make
```

- ▶ Des instructions sont souvent disponibles dans `board/<boardname>`, e.g.:
`board/raspberrypi3/readme.txt`.



Compilation



Lancer une compilation

- ▶ Aussi simple que:

```
$ make
```

- ▶ Si veut garder un log de la compilation pour analyse ou investigation:

```
$ make 2>&1 | tee build.log
```



Résultats de compilation

- ▶ Localisé dans `output/images`
- ▶ Selon la configuration, le répertoire contiendra:
 - ▶ Une ou plusieurs images de root filesystem dans des formats différents
 - ▶ Une image kernel, un ou plusieurs Device Tree blobs
 - ▶ Une ou plusieurs images de bootloader
- ▶ Pas de standard pour installer les images sur une carte
 - ▶ Trop spécifiques à la carte
 - ▶ Buildroot fournit des outils pour générer une image de carte SD / clef USB (*genimage*) ou de flasher directement sur certaines plateformes.



Organisation du build

- ▶ Toutes les sorties de compilation vont dans `output/`
- ▶ Le fichier de configuration est un `.config`
- ▶ `buildroot/`
 - ▶ `.config`
 - ▶ `arch/`
 - ▶ `package/`
 - ▶ `output/`
 - ▶ `fs/`
 - ▶ ...



Quelques tips

- ▶ Nettoyer une cible
 - ▶ Nettoyer tout le dossier d'output mais garder la configuration :

```
$ make clean
```

- ▶ Tout nettoyer dont configuration et téléchargement des sources :

```
$ make distclean
```

- ▶ Pour avoir une compilation plus verbose
 - ▶ Par défaut, beaucoup de commandes sont cachées
 - ▶ `V=1` : pour avoir toutes les commandes

```
$ make V=1
```



Dessin issu d'une photographie de Samuel Blanc

Buildroot organisation (source/build)

Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



Source tree



Source tree (1/3)

- ▶ `Makefile`
 - ▶ top-level `Makefile`, gère la configuration et orchestre le build
- ▶ `Config.in`
 - ▶ top-level `Config.in`, options générales + inclusion de tous les autres `Config.in`



Source tree (2/3)

- ▶ `toolchain/` : paquets pour générer ou utiliser une toolchain
- ▶ `system/`
 - ▶ `skeleton/` le squelette du rootfs
 - ▶ `Config.in`, options pour des fonctionnalités système comme le système de démarrage, etc.
- ▶ `linux/` : le paquet du kernel Linux
- ▶ `package/` : tous les paquets user space (2200+)
- ▶ `fs/` : Logique pour générer une image rootfs dans différents formats (`ext2`, `squashfs`, `tar`, `ubifs`, etc)
- ▶ `boot/` : paquets de bootloader (`barebox`, `uboot`, etc)



Source tree (3/3)

- ▶ `configs/`
 - ▶ configurations par défaut pour de nombreuses plateformes
 - ▶ Similaire aux `defconfigs` du kernel
 - ▶ `atmel_xplained_defconfig`, `beaglebone_defconfig`, `raspberrypi_defconfig`, etc.
- ▶ `board/`
 - ▶ Fichiers spécifiques aux boards (config kernel, patches kernel, ...)
 - ▶ Vont de paire avec un `defconfig` dans `configs/`
- ▶ `utils/` : des utilitaires pour des développeurs Buildroot
- ▶ `docs/` : documentation en AsciiDoc pouvant être de différents formats
<https://buildroot.org/downloads/manual/manual.html>



Build tree



Build tree

- ▶ `output/`
- ▶ Répertoire global
 - ▶ `build/`
 - ▶ `buildroot-config/`
 - ▶ `busybox-1.22.1/`
 - ▶ `host-pkgconf-0.8.9/`
 - ▶ `kmod-1.18/`
 - ▶ `build-time.log`
 - ▶ Lieu d'extraction de toutes les archives
 - ▶ Lieu de compilation de chaque paquet
 - ▶ Ajout en plus de fichiers de *stamp* créés par BR
 - ▶ Variable: `BUILD_DIR`



Build tree: output/target

- ▶ output/

- ▶ target/

- ▶ bin/
 - ▶ etc/
 - ▶ lib/
 - ▶ usr/bin/
 - ▶ usr/lib/
 - ▶ usr/share/
 - ▶ usr/sbin/
 - ▶ THIS_IS_NOT_YOUR_ROOT_FILESYSTEM
 - ▶ ...

- ▶ Le rootfs **cible**

- ▶ Pas complètement prêt pour la cible: permissions, fichiers device, etc
- ▶ Utilisé pour générer le rootfs final `images/`
- ▶ Variable: `TARGET_DIR`



Build tree: output/images

- ▶ `output/`
 - ▶ `images/`
 - ▶ `zImage` : Image du noyau
 - ▶ `bcm2710-rpi-3-b.dtb` : Device-tree de la RPi3
 - ▶ `rootfs.ext2` et `rootfs.ext4` : Image du rootfs au format ext2/ext4
 - ▶ `sdcard.img` : image contenant tout ! Elle sera à flasher sur la carte SD
 - ▶ Contient les images finales: kernel, bootloader, image(s) rootfs
 - ▶ Variable: `BINARIES_DIR`



Dessin issu d'une photographie de Samuel Blanc

Introduction à git

Mylène Josserand
josserand.mylene@gmail.com

© Copyright 2004-2026, Bootlin.
Creative Commons BY-SA 3.0 license.



Qu'est-ce que Git ?

- ▶ Système de contrôle de version décentralisé
- ▶ Permet de suivre les modifications du code
- ▶ Collaboratif, rapide, très utilisé (Linux, GitHub, etc.)



Initialiser un dépôt Git

```
git init
```

- ▶ Crée un dépôt vide dans le répertoire courant.
- ▶ Stocke l'historique dans le dossier '.git'.



Ajouter et valider (commit) un fichier

```
git add fichier.c
```

- ▶ 'git add' : sélectionne les fichiers à inclure
- ▶ 'git commit' : crée un point de sauvegarde

```
git commit -m "Ajout du fichier principal"
```



Voir l'historique des commits

`git log`

- ▶ Affiche les messages de commit, auteurs, dates

`git show`

- ▶ Affiche les modifications du dernier commit



Les patches

- ▶ Un fichier texte qui contient des modifications de code.
- ▶ Il montre ce qui a été ajouté, modifié ou supprimé.
- ▶ Utilisé pour partager des changements sans envoyer tout le projet (ex. mailing list, Buildroot)
- ▶ Est la base de beaucoup de contributions open-source `git format-patch -1`
- ▶ Génère un fichier '.patch' du dernier commit `git am ma_modif.patch`
- ▶ Applique un patch en gardant l'historique (auteur, message, ...)



Bonnes pratiques

- ▶ Un commit = une modification logique
- ▶ Commits propres et lisibles
- ▶ Utiliser 'git diff', 'git log' pour valider les changements
- ▶ Préférer 'git format-patch' + 'git am' pour les contributions sérieuses



Conclusion

- ▶ **Bootloader** : initialisation de la RAM + chargement du kernel [*u-boot* ; *barebox*]
- ▶ **Kernel** : Gère le matériel, les ressources, protocoles, etc
- ▶ **Userspace** : Rootfs avec LibC, Init, packages (i.e. applications, librairies, etc)x
- ▶ **Build system** : Buildroot ou Yocto
- ▶ Logiciels libres:
 - ▶ Facilite le debug
 - ▶ Facilite le développement
 - ▶ Contrôle total sur le système
 - ▶ Communauté en support
 - ▶ Logiciels testés et corrigés par des millions de personnes

Des questions ? Sinon, travaux pratiques !